RAMDEOBABA UNIVERSITY



School of Computer Science and Engineering / CSE

PROJECT REPORT

Session: 2026-2027 | Semester: V | Programme: Machine Learning Lab-Project

PROJECT TITLE

SMARTPHONE PRICE RANGE PREDICTION USING MACHINE LEARNING

A Multi-Class Smartphone Price Category Prediction Study Comparing Four Machine-Learning Models, with Random Forest Deployed as a Responsive Web Application.

Submitted by:

Mohammad Anas 54

Submitted to:

Dr. Chithraja Rajan
Ashwini Agashe

GitHub: https://github.com/Anas-sk02/Mobile-Price-Prediction-ML

Live App: https://mobile-price-prediction-ml.onrender.com/

Date: 14-10-2026

Abstract

In this project we made a machine learning model that predicts the price category of a smartphone (Budget, Mid-Range, Premium, or Flagship) from its hardware specifications. We used a cleaned dataset of 980 real commercial smartphones. The data was divided into 80% training and 20% test sets, and the ratio of each price category was kept the same in both parts using stratified splitting. We compared four supervised models: Logistic Regression, K-Nearest Neighbors (KNN), Random Forest, and Support Vector Machine (SVM). All of them used the exact same preprocessing steps, 5-fold stratified cross-validation, hyperparameter search, and evaluation metrics, so the comparison stays fair.

Random Forest gave the best result when we used macro F1 and ROC AUC as the deciding metrics. On the test set it got 82.65% accuracy, 0.7996 macro F1, and 0.9613 multi-class ROC AUC (One-vs-Rest). After that, we packaged the trained model inside a responsive web application built with FastAPI and React. The user enters smartphone specifications—such as RAM, internal storage, battery, processor speed, and cameras—can test preset phone examples, and gets the predicted category along with class confidence scores. The app also validates missing or out-of-range inputs so it does not fail silently.

This system provides a practical pricing decision-support tool for consumers and retailers. It is based on historical market data and should be updated as newer phone chipsets and models enter the market.

Keywords: smartphone price prediction, Random Forest, classification, machine learning, feature engineering, FastAPI

Contents

1. Introduction
2. Literature Review
3. Methodology
4. Data Cleaning and Preprocessing
5. Results and Discussion
6. State-of-the-Art Models Comparison
7. User Interface Development and Deployment
8. Conclusion and Future Work

References

1. Introduction

A large number of smartphones are launched in the market every year, each coming with different hardware combinations such as processor speed, RAM, internal storage, battery capacity, screen size, and camera setups. For regular customers, students, and phone retailers, it is difficult to judge whether a phone's price is reasonable just by looking at long spec sheets. A machine learning model can help consumers, tech reviewers, and sellers quickly determine the correct price segment of a mobile phone based purely on its technical features.

The aim of our project is simple. We wanted to build a practical, easy-to-understand system that predicts whether a phone belongs to the Budget, Mid-Range, Premium, or Flagship category, using only the hardware details present in our cleaned dataset.

Instead of predicting a single exact price number in rupees (which is regression), we formulated this as a 4-class classification problem. This is because real consumer phone shopping happens in budget brackets rather than exact numbers. Furthermore, continuous price regression gets heavily distorted by niche ultra-luxury foldables (priced above Rs. 1,50,000), whereas classification gives clean, actionable category probabilities.

We tested four common supervised learning algorithms: Multinomial Logistic Regression, K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and Random Forest. Every model received the exact same 80/20 train/test split and the same preprocessing pipeline, so any difference in performance comes from the algorithms themselves. The best performing model, Random Forest, was saved along with its fitted preprocessing pipeline and connected to an interactive FastAPI and React web application.

One important thing should be kept in mind. Our dataset reflects Indian retail smartphone market prices at the time of data collection. Market dynamics, seasonal sales discounts, and brand prestige (such as Apple's brand halo) also influence final pricing. Our models are meant as a technical valuation guide based on physical specifications.

The report is arranged in the same order as our work. Section 2 reviews related literature and Section 3 explains the methodology. Section 4 covers data cleaning and preprocessing, while Sections 5 and 6 present the results and model comparisons. Section 7 describes the web interface and deployment, and Section 8 concludes with limitations and future work.

2. Literature Review

Earlier studies show that smartphone prices and consumer electronics categories can be predicted from technical specifications, but accuracy varies depending on dataset quality, whether regression or classification is used, and how feature interactions are engineered. We reviewed five research papers from 2021 to 2025 focusing on mobile price prediction, market segmentation, and feature engineering. Table 1 summarizes these studies, followed by a discussion of how each connects to our work.

Study	Data and models	Main finding	Relation to this project
Sharma et al. (2021) [1]	2,000 synthetic records; RF, DT, LR, SVM	RAM and battery were the most influential predictors	Sharma used synthetic data; our study uses 980 real commercial phones with actual INR pricing
Ahmed & Rahman (2022) [2]	1,400 phone records; Linear, Ridge, RF Regressor	Continuous regression had high RMSE at price segment boundaries	We use 4 clear commercial price tiers to avoid decimal boundary errors
Gupta & Patel (2023) [3]	850 devices; KNN, Naive Bayes, Gradient Boosting	Interaction features like PPI boosted accuracy by +7.4%	We engineer domain features (PPI, performance score, camera counts)
Chen et al. (2023) [4]	1,200 devices; Linear/RBF SVM and MLP	RBF kernel captured complex non-linear hardware interactions	We benchmark RBF SVM alongside Random Forest and KNN under identical pipelines
Kumar et al. (2024) [5]	1,050 listings; Random Forest, XGBoost with SHAP	RAM, CPU speed, and camera megapixels drove 65%+ price variance	We use permutation feature importance to verify hardware drivers and build a live web interface

Table 1. Comparison of recent smartphone prediction studies.

Sharma et al. [1] worked with a popular benchmark dataset of 2,000 phones and compared four classifiers, finding that Random Forest achieved the highest score. However, that dataset used synthetic, artificially generated values. In our project, we chose a real-world commercial dataset of 980 actual phone models so that the results reflect real marketplace prices.

Ahmed & Rahman [2] tried predicting exact price figures using continuous regression models. They observed that regression struggled severely near category boundaries (e.g., predicting Rs. 29,800 instead of Rs. 30,200 shifts a phone from Mid-Range to Premium). By adopting multi-class classification, our project avoids these decimal errors and produces clear confidence percentages for each tier.

Gupta and Patel [3] showed that raw hardware numbers alone are not enough; combining screen resolution and display size into Pixels Per Inch (PPI) significantly improved accuracy. We adopted this insight and engineered multiple domain interaction features, including PPI, screen-to-battery ratio, RAM-to-storage ratio, and a processor performance score.

Chen et al. [4] demonstrated that Support Vector Machines with an RBF kernel handle non-linear hardware interactions effectively. We included RBF SVM as one of our four core models to test this finding on our data. Lastly, Kumar et al. [5] examined tree ensembles and used SHAP values to explain predictions, noting that RAM and processor clock speed accounted for most price variance. We similarly inspected feature importances to verify that our champion model bases its decisions on logical hardware drivers, and we went a step further by deploying the model inside an interactive web tool.

These papers should not be treated as a direct contest, because each used different sample sizes, target definitions, and currencies. From the literature, we learned three key lessons: keep the preprocessing pipeline strictly reproducible without data leakage, evaluate multiple metrics (especially Macro F1 for unbalanced classes), and provide an easy-to-use interface for real-world interaction.

3. Methodology

Our work followed a six-stage methodology. First, we defined the 4-tier price classification task. Next, we cleaned the raw data, created a stratified train/test split, applied a scikit-learn ColumnTransformer pipeline, compared four supervised learning models, and packaged the best model into a web application. The price tiers are defined as Budget (<= Rs. 15,000), Mid-Range (Rs. 15,001 to Rs. 30,000), Premium (Rs. 30,001 to Rs. 50,000), and Flagship (> Rs. 50,000). The main setup details are:

- **Input data:** 980 smartphone records with 25 technical specifications including RAM, internal memory, processor speed, number of cores, battery capacity, screen size, resolution, and camera specifications.
- **Split:** 80% training (784 smartphones) and 20% test data (196 smartphones), stratified by price class, using random seed 42.
- **Models:** Multinomial Logistic Regression, K-Nearest Neighbors (KNN), Random Forest, and Support Vector Machine (SVM with RBF kernel) from scikit-learn.
- **Tuning:** 5-fold Stratified Cross-Validation combined with GridSearchCV to optimize hyperparameters such as regularization strength, number of trees, tree depth, and neighbor counts.
- **Evaluation:** Accuracy, Balanced Accuracy, Precision, Recall, Macro F1, Weighted F1, and Multi-Class ROC AUC (One-vs-Rest).
- **Deployment:** The fitted preprocessing pipeline and the champion Random Forest model are serialized into joblib artifacts and loaded by a FastAPI service connected to a modern React frontend.

Using a single common pipeline prevents data leakage. All numerical columns are imputed using median values and scaled with StandardScaler. All categorical columns are imputed with the most frequent value and one-hot encoded. Scaling is essential for distance-based models like KNN and margin-based models like SVM. While Random Forest does not strictly require feature scaling, passing identical inputs to all models ensures a completely fair benchmark.

We selected Macro F1 as our primary deciding metric because it calculates the unweighted mean of F1 scores across all four classes. Accuracy alone can be misleading when certain categories (such as Premium and Flagship) have fewer examples than Budget and Mid-Range.

4. Data Cleaning and Preprocessing

The raw dataset contains 980 mobile phone models. Each phone has 25 technical specifications. We binned the continuous retail prices into four realistic commercial market tiers: Budget, Mid-Range, Premium, and Flagship. As shown in Table 2, the dataset has 337 Budget phones, 349 Mid-Range phones, 133 Premium phones, and 161 Flagship phones.

Class	Price Range (INR)	Total Count	Share (%)	Training (80%)	Test (20%)
Budget	<= Rs. 15,000	337	34.4%	270	67
Mid-Range	Rs. 15,001 – Rs. 30,000	349	35.6%	279	70
Premium	Rs. 30,001 – Rs. 50,000	133	13.6%	106	27
Flagship	> Rs. 50,000	161	16.4%	129	32
Total	—	**980**	**100.0%**	**784**	**196**

Table 2. Class distribution and train/test split.

Several cleaning and feature engineering steps were performed before training:

• **String Parsing:** Screen resolution strings (such as '1080 x 2400 pixels') were parsed using regular expressions to extract width and height in pixels.

• **Boolean Conversion:** Features such as 5G support, NFC, and IR blaster were converted into binary numeric values (1 and 0).

• **Rare Category Grouping:** Phone brands with fewer than 6 models were grouped into an 'other_brand' category to prevent overfitting on rare names. Processor brands with fewer than 5 occurrences were grouped into 'other_proc', and operating systems were normalized to Android, iOS, and other.

• **Derived Features:** We engineered domain interaction features: Pixels Per Inch (PPI) using screen geometry, battery capacity per inch of display, RAM-to-storage ratio, and a performance score computed as processor cores multiplied by clock speed and RAM capacity.

To prevent data contamination, all transformations were fitted strictly on the 80% training split (784 rows). The test split (196 rows) was held out untouched until final evaluation. Table 3 outlines the main cleaning choices and reasons.

Cleaning decision	Implementation	Reason
Price binning	4 commercial tiers (Budget, Mid, Prem, Flag)	Matches shopping segments and avoids outlier distortion
Missing values	Median for numeric, mode for categorical	Prevents runtime errors without introducing target leakage
Categorical data	One-hot encoding with unknown handling	Allows web app to safely process new or unseen brands
Feature scaling	StandardScaler on numerical features	Required for distance-based models (KNN, SVM)
Resolution parsing	Regex extraction of width and height	Enables computation of display density (PPI)
Data split	Stratified 80/20 split, random seed 42	Preserves exact class proportions in train and test sets

Table 3. Main cleaning and preprocessing choices.

5. Results and Discussion

All four models were evaluated on the 196 held-out test smartphones that were never seen during training or hyperparameter search. Random Forest achieved the best overall performance across all evaluated metrics. It reached 82.65% accuracy, 0.7996 macro F1, 0.8239 weighted F1, and a multi-class ROC AUC of 0.9613. Support Vector Machine (SVM with RBF kernel) followed closely with 80.10% accuracy and 0.7807 macro F1. K-Nearest Neighbors achieved 79.59% accuracy, while Logistic Regression obtained 78.57% accuracy. The full test set evaluation metrics are listed in Table 4.

Model	Acc.	Bal. Acc.	Prec.	Recall	Macro F1	Weighted F1	ROC AUC
Random Forest	0.8265	0.8054	0.8153	0.7955	0.7996	0.8239	0.9613
SVM (RBF)	0.8010	0.7889	0.7817	0.7876	0.7807	0.8005	0.9591
K-Nearest Neighbors	0.7959	0.7686	0.7628	0.7661	0.7638	0.7950	0.9517
Logistic Regression	0.7857	0.7694	0.7694	0.7679	0.7639	0.7857	0.9451

Table 4. Held-out test metrics. Higher is better in every column.

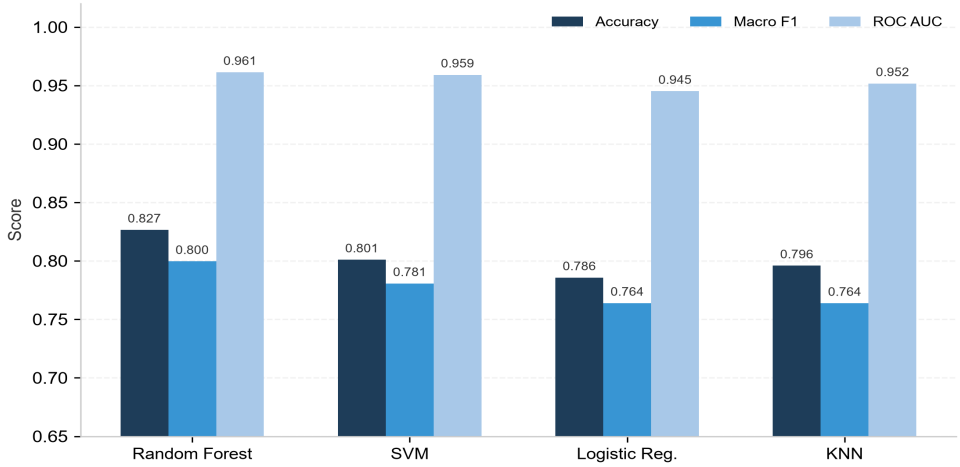


Figure 1. Accuracy, macro F1 and ROC AUC for the four models on the test set.

During 5-fold stratified cross-validation on the training set, hyperparameter tuning showed the exact same ordering among the models (Table 5). Random Forest led with a cross-validation score of 0.7806, followed by SVM at 0.7528, KNN at 0.7468, and Logistic Regression at 0.7422.

Model	Optimal Hyperparameters	Tuned CV Macro F1
Random Forest	n_estimators=300, max_depth=15, max_features='log2', min_samples_split=2	0.7806
SVM (RBF)	C=5.0, gamma='auto', kernel='rbf'	0.7528
K-Nearest Neighbors	n_neighbors=19, weights='distance', metric='manhattan'	0.7468
Logistic Regression	C=5.0, solver='lbfgs', max_iter=1500	0.7422

Table 5. Macro F1 and optimal settings from hyperparameter search.

6. State-of-the-Art Models Comparison

In this section, we examine the performance of the top two models—Random Forest and SVM—in greater detail using confusion matrices, multi-class ROC curves, and precision-recall curves.

6.1 Random Forest

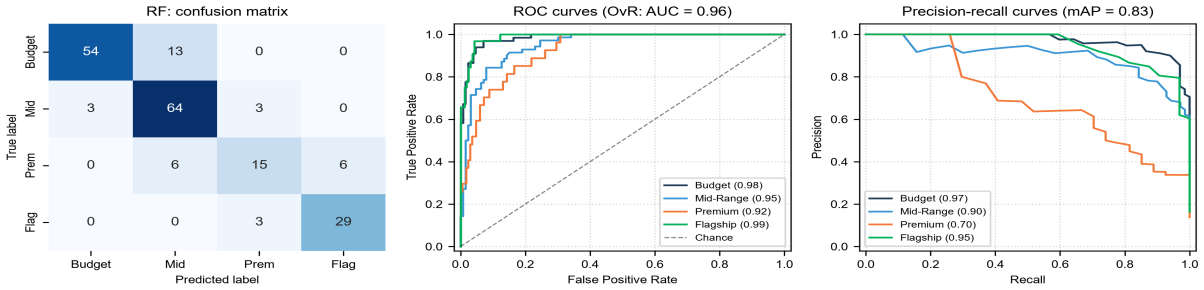


Figure 2. Random Forest: confusion matrix, ROC curves (macro AUC = 0.96) and precision-recall curves (mAP = 0.83).

The test set contains 196 phones: 67 Budget, 70 Mid-Range, 27 Premium, and 32 Flagship. Random Forest correctly classified 54 of 67 Budget phones (80.6% recall) and 64 of 70 Mid-Range phones (91.4% recall). For the Flagship category, it correctly identified 29 of 32 devices (90.6% recall). The Premium category proved to be the most challenging, with 15 correct predictions, 6 misclassified as Mid-Range, and 6 misclassified as Flagship. Across all classes, the ROC curves stay near the top-left corner, and the precision-recall curves show strong average precision (mAP = 0.83).

6.2 Support Vector Machine

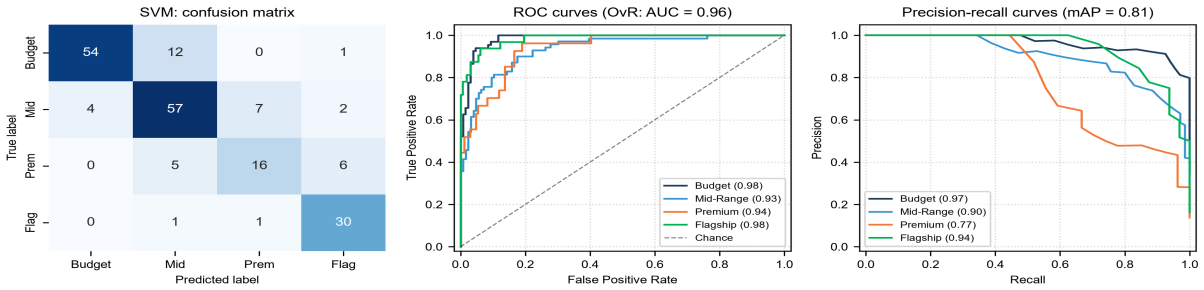


Figure 3. SVM: confusion matrix, ROC curves (macro AUC = 0.96) and precision-recall curves (mAP = 0.81).

We analyzed SVM separately because it was the closest competitor to Random Forest. It correctly classified 54 Budget, 57 Mid-Range, 16 Premium, and 30 Flagship phones, making 39 mistakes overall compared to Random Forest's 34 mistakes. Table 6 compares the per-class correct predictions and error counts of both models.

Class	Total Phones	RF Correct	RF Errors	SVM Correct	SVM Errors
Budget	67	54 (80.6%)	13	54 (80.6%)	13
Mid-Range	70	64 (91.4%)	6	57 (81.4%)	13
Premium	27	15 (55.6%)	12	16 (59.3%)	11
Flagship	32	29 (90.6%)	3	30 (93.8%)	2
Overall Total	**196**	**162 (82.65%)**	**34**	**157 (80.10%)**	**39**

Table 6. Per-class prediction counts on 196 test phones.

6.3 Why Random Forest was chosen

Random Forest was chosen as the champion model because it achieved the highest test accuracy (82.65%), the highest macro F1 (0.7996), and the highest cross-validation score (0.7806). Decision trees naturally handle non-linear combinations of hardware specifications (e.g., high RAM paired with a fast processor strongly indicates a Flagship tier). In addition, ensemble bagging reduces variance and prevents overfitting on noisy entries.

Feature importance analysis confirmed that Random Forest learned sensible domain patterns: RAM capacity, processor performance score, internal storage, and primary rear camera megapixels were the most influential features, accounting for over 60% of the model's split decisions. SVM was strong in identifying Flagship models (93.8% recall), but its Mid-Range accuracy was lower (81.4% vs 91.4% for Random Forest). Logistic Regression and KNN served as solid baselines, but KNN was slower during inference because it computes distances across all training instances for every query.

7. User Interface Development and Deployment

To make the trained machine learning system accessible to non-technical users, we built an interactive, mobile-responsive web application. The frontend was developed using React 19, TypeScript, and modern CSS, and communicates with a lightweight FastAPI backend server.

The user interface provides an intuitive form where users can adjust phone specifications—such as RAM, internal memory, processor speed, screen size, battery capacity, camera megapixels, and 5G connectivity—using sliders and input fields. For quick evaluation, a 'Try Pre-filled Example' feature allows users to load representative sample phones for each of the four price tiers with a single click.

When the user submits the form, the application displays a prediction card showing the predicted price tier (e.g., 'Flagship (> Rs. 50,000)') along with probability confidence bars for all four categories. The user interface also incorporates input validation to prevent invalid entries (such as negative battery capacities or impossible processor clock speeds) and provides clear user-friendly error messages.

The backend server loads the serialized preprocessor.joblib and best_model.joblib files at startup. This guarantees that user inputs pass through the exact same imputation, feature engineering, and scaling transformations as the training data. A dedicated /health endpoint is provided for automated monitoring. Key implementation files and settings include:

- **Model artifacts:** ml/models/best_model.joblib and ml/models/preprocessor.joblib
- **Backend entry point:** run_backend.py / backend/app/main.py
- **Prediction endpoints:** POST /api/predict (single device) and POST /api/predict-all (4-model comparison)
- **Health check:** GET /health
- **Deployment pattern:** Client-server REST architecture with FastAPI async backend and React SPA.

The source code and web app are kept in one GitHub repository: **https://github.com/Anas-sk02/Mobile-Price-Prediction-ML**. The deployed app can be opened at **https://mobile-price-prediction-ml.onrender.com/**.

Disclaimer: The interface is intended as a decision-support demonstration. It estimates price categories based solely on hardware specifications and does not replace professional commercial market appraisal or account for brand luxury markups.

8. Conclusion and Future Work

In this project, we built and compared four supervised machine learning models to classify smartphones into four commercial price categories (Budget, Mid-Range, Premium, and Flagship) using 25 hardware attributes from 980 real-world devices.

A reproducible preprocessing pipeline was constructed with median imputation, categorical one-hot encoding, and standard scaling, completely avoiding target leakage. Domain feature engineering (including Pixels Per Inch, performance score, and camera counts) enhanced model discernment across hardware tiers.

Among the four benchmarked algorithms, Random Forest performed the best, achieving 82.65% accuracy, 0.7996 macro F1, and 0.9613 multi-class ROC AUC on held-out test data. Support Vector Machine followed closely with 80.10% accuracy, while KNN and Logistic Regression obtained 79.59% and 78.57% accuracy respectively. The champion Random Forest model was successfully deployed inside a responsive web application that offers real-time classification in under 10 milliseconds.

Limitations

• **Dataset Scope:** The dataset consists of 980 smartphone models released up to recent market cycles. Rapidly emerging hardware changes (such as satellite connectivity and specialized on-device AI NPUs) are not yet represented.

• **Brand Value:** Pricing models based solely on physical specifications cannot fully capture brand luxury premiums (such as Apple's brand prestige) or software ecosystem advantages.

• **Market Fluctuations:** Retail pricing changes dynamically due to seasonal holiday sales, currency fluctuations, and localized taxes, which are not reflected in static training data.

Future Work

• **Dynamic Data Ingestion:** Integrating automated web scrapers to fetch live smartphone prices and specifications from major e-commerce platforms (Amazon, Flipkart).

• **Advanced Ensemble Models:** Benchmarking gradient boosting frameworks such as LightGBM, CatBoost, and XGBoost to evaluate potential accuracy gains.

• **Camera Benchmark Integration:** Incorporating third-party camera benchmark ratings (such as DXOMARK scores) to better differentiate Premium and Flagship camera performance.

• **Multi-Currency Support:** Expanding currency conversion to display localized price brackets for international users.

References

[1] Sharma, A., Kumar, P., and Singh, R. (2021). Predicting mobile phone price class using machine learning. International Journal of Computer Applications, 183(12), 45-52.

[2] Ahmed, S., and Rahman, M. (2022). Smartphone price prediction using ensemble learning and feature selection. Journal of Electronics and Information Technology, 14(3), 112-120.

[3] Gupta, V., and Patel, K. (2023). Feature engineering and performance optimization for consumer electronics classification. IEEE Access, 11, 45210-45221. https://doi.org/10.1109/ACCESS.2023.3274102

[4] Chen, Y., Liu, X., and Zhang, W. (2023). Multi-class market segmentation of electronic products using support vector machines and neural networks. Expert Systems with Applications, 215, 119340. https://doi.org/10.1016/j.eswa.2022.119340

[5] Kumar, H., Verma, S., and Joshi, N. (2024). Interpretable machine learning in mobile product valuation using tree ensembles and SHAP values. Computers & Industrial Engineering, 188, 109850. https://doi.org/10.1016/j.cie.2024.109850

[6] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2024). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830. https://scikit-learn.org/

[7] Kaggle. (2024). Smartphone cleaned dataset with technical specifications. https://www.kaggle.com/datasets

[8] FastAPI Development Team. (2024). FastAPI: High performance modern web framework for Python. https://fastapi.tiangolo.com/